



HOA SEN
UNIVERSITY

Lecture 11

The Relational Algebra and Relational Calculus – 2

Objectives

- Relational Calculus
 - Tuple Relational Calculus
 - Domain Relational Calculus

- Reference: Chapter 8

Unary Relational Operations: SELECT Operation Properties

- SELECT Operation Properties

- The SELECT operation $\sigma_{\langle \text{selection condition} \rangle}(R)$ produces a relation S that has the same schema (same attributes) as R
- SELECT σ is commutative:
 - $\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$
- Conjunction of all the conditions:
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \langle \text{cond3} \rangle}(R)$
- The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R

Unary Relational Operations: PROJECT Operation Properties

- PROJECT Operation Properties
 - The number of tuples in the result of projection $\pi_{\langle \text{list} \rangle}(R)$ is always less or equal to the number of tuples in R
 - If the list of attributes includes a *key* of R , then the number of tuples in the result of PROJECT is *equal* to the number of tuples in R
 - PROJECT is *not* commutative
 - $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$ as long as $\langle \text{list2} \rangle$ contains the attributes in $\langle \text{list1} \rangle$

Some properties of UNION, INTERSECT, and DIFFERENCE

- Notice that both union and intersection are *commutative* operations; that is
 - $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are *associative* operations; that is
 - $R \cup (S \cup T) = (R \cup S) \cup T$
 - $(R \cap S) \cap T = R \cap (S \cap T)$
- The minus operation is *not commutative*; that is, in general
 - $R - S \neq S - R$

Some properties of JOIN

- Consider the following JOIN operation:
 - $R(A_1, A_2, \dots, A_n) \bowtie_{R.A_i=S.B_j} S(B_1, B_2, \dots, B_m)$
 - Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
 - The resulting relation state has one tuple for each combination of tuples — r from R and s from S , but *only if they satisfy the join condition* $r[A_i]=s[B_j]$
 - Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R * n_S$ tuples.
 - Only related tuples (based on the join condition) will appear in the result

Some properties of JOIN (2)

- The general case of JOIN operation is called a Theta-join: $R \theta S$
 - The join condition is called *theta*
- *Theta* can be any general boolean expression on the attributes of R and S; for example:
 - $R.A_i < S.B_j \text{ AND } (R.A_k = S.B_l \text{ OR } R.A_p < S.B_q)$
- Most join conditions involve one or more equality conditions “AND”ed together; for example:
 - $R.A_i = S.B_j \text{ AND } R.A_k = S.B_l \text{ AND } R.A_p = S.B_q$

- DIVISION Operation

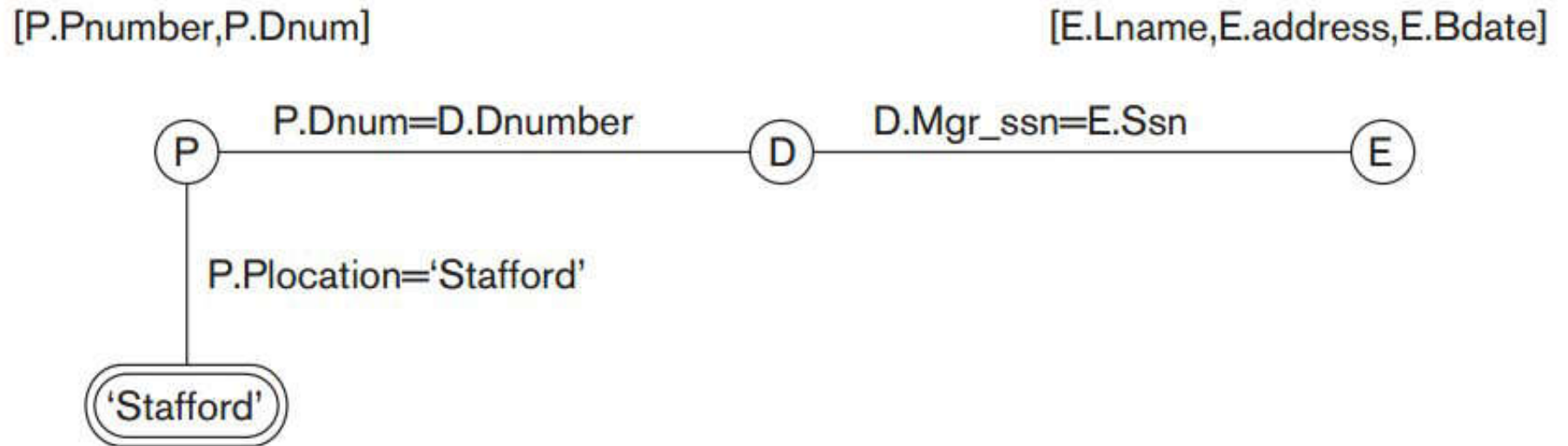
- The division operation is applied to two relations
- $R(Z) \div S(X)$, where $X \subset Z$. Let $Y = Z - X$ (and hence $Z = X \cup Y$); that is, let Y be the set of attributes of R that are not attributes of S .
- The result of DIVISION is a relation $T(Y)$ that includes a tuple t if tuples t_R appear in R with $t_R[Y] = t$, and with
 - $t_R[X] = t_s$ for every tuple t_s in S .
- For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with every tuple in S .

Other Example

- For every project located in 'Stafford', retrieve the project number, the controlling department number, and the department manager's last name, address, and birth date:

$\pi_{Pnumber, Dnum, Lname, Address, Bdate}(((\sigma_{Plocation='Stafford'}(PROJECT)) \bowtie$
 $Dnum=Dnumber(DEPARTMENT)) \bowtie Mgr_ssn=Snn(EMPLOYEE))$

Query Graph



Additional Relational Operations: Aggregate Functions and Grouping

- A type of request that cannot be expressed in the basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database.
- Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples.
 - These functions are used in simple statistical queries that summarize information from the database tuples.
- Common functions applied to collections of numeric values include
 - SUM, AVERAGE, MAXIMUM, and MINIMUM.
- The COUNT function is used for counting tuples or values.

Aggregate Function Operation

- Use of the Aggregate Functional operation $\mathcal{F}$
 - $\mathcal{F}_{\text{MAX Salary}}(\text{EMPLOYEE})$ retrieves the maximum salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{MIN Salary}}(\text{EMPLOYEE})$ retrieves the minimum Salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$ retrieves the sum of the Salary from the EMPLOYEE relation
 - $\mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$ computes the count (number) of employees and their average salary
 - Note: count just counts the number of rows, without removing duplicates

Using Grouping with Aggregation

- The previous examples all summarized one or more attributes for a set of tuples
 - Maximum Salary or Count (number of) Ssn
- Grouping can be combined with Aggregate Functions
- **Example:** For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY
- A variation of aggregate operation $\mathcal{F}$ allows this:
 - Grouping attribute placed to left of symbol
 - Aggregate functions to right of symbol
 - $\text{DNO } \mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}} (\text{EMPLOYEE})$
- Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

Additional Relational Operations (2)

- Recursive Closure Operations
 - Another type of operation that, in general, cannot be specified in the basic original relational algebra is **recursive closure**.
 - This operation is applied to a **recursive relationship**.
 - An example of a recursive operation is to retrieve all SUPERVISEES of an EMPLOYEE e at all levels — that is, all EMPLOYEE e' directly supervised by e ; all employees e'' directly supervised by each employee e' ; all employees e''' directly supervised by each employee e'' ; and so on.

Additional Relational Operations: Outer Join

- In NATURAL JOIN and EQUIJOIN, tuples without a *matching* (or *related*) tuple are eliminated from the join result
 - Tuples with null in the join attributes are also eliminated
 - This amounts to loss of information.
- A set of operations, called OUTER joins, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation.

Additional Relational Operations: Outer Join (2)

- The left outer join operation keeps every tuple in the first or left relation R in $R \bowtie_{\text{L}} S$; if no matching tuple is found in S , then the attributes of S in the join result are filled or “padded” with null values.
- A similar operation, right outer join, keeps every tuple in the second or right relation S in the result of $R \bowtie_{\text{R}} S$.
- A third operation, full outer join, denoted by $\bowtie_{\text{F}}$ keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

Additional Relational Operations: Outer Union

- The outer union operation was developed to take the union of tuples from two relations if the relations are *not type compatible*.
- This operation will take the union of tuples in two relations $R(X, Y)$ and $S(X, Z)$ that are **partially compatible**, meaning that only some of their attributes, say X , are type compatible.
- The attributes that are type compatible are represented only once in the result, and those attributes that are not type compatible from either relation are also kept in the result relation $T(X, Y, Z)$.

Q1: Retrieve the name and address of all employees who work for the 'Research' department.

$\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{DNAME}='Research'} (\text{DEPARTMENT})$

$\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{DNUMBER}=\text{DNO}} \text{EMPLOYEE})$

$\text{RESULT} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{ADDRESS}} (\text{RESEARCH_EMPS})$

Q2: Retrieve the names of employees who have no dependents.

$ALL_EMPS \leftarrow \pi_{SSN}(EMPLOYEE)$

$EMPS_WITH_DEPS(SSN) \leftarrow \pi_{ESSN}(DEPENDENT)$

$EMPS_WITHOUT_DEPS \leftarrow (ALL_EMPS - EMPS_WITH_DEPS)$

$RESULT \leftarrow \pi_{LNAME, FNAME}(EMPS_WITHOUT_DEPS * EMPLOYEE)$

Relational Calculus

- A **relational calculus** expression creates a new relation, which is specified in terms of variables that range over rows of the stored database relations (in **tuple calculus**) or over columns of the stored relations (in **domain calculus**).
- In a calculus expression, there is *no order of operations* to specify how to retrieve the query result—a calculus expression specifies only what information the result should contain.
 - This is the main distinguishing feature between relational algebra and relational calculus.

Relational Calculus (2)

- Relational calculus is considered to be a **nonprocedural** language.
- This differs from relational algebra, where we must write a *sequence of operations* to specify a retrieval request; hence relational algebra can be considered as a **procedural** way of stating a query.

Tuple Relational Calculus

- The tuple relational calculus is based on specifying a number of tuple variables.
- Each tuple variable usually ranges over a particular database relation, meaning that the variable may take as its value any individual tuple from that relation.
- A simple tuple relational calculus query is of the form
 $\{t \mid \text{COND}(t)\}$
 - where t is a tuple variable and $\text{COND}(t)$ is a conditional expression involving t .
 - The result of such a query is the set of all tuples t that satisfy $\text{COND}(t)$.

Tuple Relational Calculus (2)

- Example: To find the first and last names of all employees whose salary is above \$50,000, we can write the following tuple calculus expression:

$\{t.Fname, t.Lname \mid EMPLOYEE(t) \text{ AND } t.Salary > 50000\}$

- The condition $EMPLOYEE(t)$ specifies that the **range relation** of tuple variable t is $EMPLOYEE$.
- The first and last name (PROJECTION $\pi_{Fname, Lname}$) of each $EMPLOYEE$ tuple t that satisfies the condition $t.SALARY > 50000$ (SELECTION $\sigma_{Salary > 50000}$) will be retrieved

The Existential and Universal Quantifiers

- Two special symbols called quantifiers can appear in formulas; these are the universal quantifier ($\forall$) and the existential quantifier ($\exists$).
- Informally, a tuple variable t is bound if it is quantified, meaning that it appears in an $(\forall t)$ or $(\exists t)$ clause; otherwise, it is free.
- If F is a formula, then so are $(\exists t)(F)$ and $(\forall t)(F)$, where t is a tuple variable.
 - The formula $(\exists t)(F)$ is true if the formula F evaluates to true for some (at least one) tuple assigned to free occurrences of t in F ; otherwise $(\exists t)(F)$ is false.
 - The formula $(\forall t)(F)$ is true if the formula F evaluates to true for every tuple (in the universe) assigned to free occurrences of t in F ; otherwise $(\forall t)(F)$ is false.

The Existential and Universal Quantifiers (2)

- $\forall$ is called the universal or “for all” quantifier because every tuple in “the universe of” tuples must make F true to make the quantified formula true.
- $\exists$ is called the existential or “there exists” quantifier because any tuple that exists in “the universe of” tuples may make F true to make the quantified formula true.

Languages Based on Tuple Relational Calculus

- The language **SQL** is based on tuple calculus. It uses the basic block structure to express the queries in tuple calculus:

```
SELECT <list of attributes>  
FROM <list of relations>  
WHERE <conditions>
```

- SELECT clause mentions the attributes being projected, the FROM clause mentions the relations needed in the query, and the WHERE clause mentions the selection as well as the join conditions.
 - SQL syntax is expanded further to accommodate other operations.

The Domain Relational Calculus

- Another variation of relational calculus called the domain relational calculus, or simply, domain calculus is equivalent to tuple calculus and to relational algebra.
- The language called QBE (Query-By-Example) that is related to domain calculus was developed almost concurrently to SQL at IBM Research, Yorktown Heights, New York.
 - Domain calculus was thought of as a way to explain what QBE does.
- Domain calculus differs from tuple calculus in the type of variables used in formulas:
 - Rather than having variables range over tuples, the variables range over single values from domains of attributes.
- To form a relation of degree n for a query result, we must have n of these domain variables — one for each attribute.

The Domain Relational Calculus (2)

- An expression of the domain calculus is of the form $\{ \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \mid \mathbf{COND}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n, \mathbf{x}_{n+1}, \mathbf{x}_{n+2}, \dots, \mathbf{x}_{n+m}) \}$
 - where $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n, \mathbf{x}_{n+1}, \mathbf{x}_{n+2}, \dots, \mathbf{x}_{n+m}$ are domain variables that range over domains (of attributes)
 - and COND is a condition or formula of the domain relational calculus.

Example Query Using Domain Calculus

- Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.
- Query :
$$\{uv \mid (\exists q) (\exists r) (\exists s) (\exists t) (\exists w) (\exists x) (\exists y) (\exists z)$$

$$(\text{EMPLOYEE}(qrstuvwxyz) \text{ and } q=\text{'John'} \text{ and } r=\text{'B'} \text{ and } s=\text{'Smith'})\}$$
- Ten variables for the employee relation are needed, one to range over the domain of each attribute in order.
 - Of the ten variables $q, r, s, \dots, z$, only u and v are free.
- Specify the *requested attributes*, Bdate and Address, by the free domain variables u for Bdate and v for Address
- Specify the condition for selecting a tuple following the bar ($\mid$) —
 - namely, that the sequence of values assigned to the variables $qrstuvwxyz$ be a tuple of the employee relation and that the values for q (Fname), r (Minit), and s (Lname) be 'John', 'B', and 'Smith', respectively.

Q & A

